Real-Time Search Results Streaming Implementation Plan

Revision: 1 **Last modified:** 2026-06-06T00:00:00Z

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Stream search results to users in real-time as they arrive from each tracker, using SSE endpoint for live updates

Architecture: Modify SearchOrchestrator to yield intermediate results as they arrive. Update SSE endpoint to read from individual results and emit incrementally. Update frontend to use SSE endpoint and EventSource for real-time UI updates.

Tech Stack: Python asyncio, FastAPI StreamingResponse, Server-Sent Events (SSE), JavaScript EventSource

---

Task 1: Write failing tests for streaming individual results

Files: - Modify: `tests/unit/test_streaming.py` - Modify: `tests/integration/test_ui_quick.py`

- [ ] **Step 1: Write failing tests** - Add tests that verify individual results stream in real-time

```
def test_streaming_yields_individual_results(self):
    """Test that search_results_stream yields individual results as they a
    # Test that streaming emits results_update events with actual result d
    # NOT just count changes - should include result details
```

- [ ] **Step 2: Run tests to verify they fail**

Run: `pytest tests/unit/test_streaming.py -v -k "individual"` Expected: FAIL (tests expect feature that doesnâ€™t exist yet)

Task 2: Modify backend to emit incremental results

Files: - Modify: `download-proxy/src/merge_service/search.py` (lines 285-310) - Modify: `download-proxy/src/api/streaming.py` (lines 53-110)

- [ ] **Step 1: Add incremental results tracking to SearchOrchestrator**

Add method to track results as they arrive from each tracker:

```
def get_live_results(self, search_id: str) -> List[SearchResult]:
    """Get all results found so far for a search, not yet merged."""
    if search_id not in self._tracker_results:
        return []
    results = []
    for tracker_results in self._tracker_results[search_id].values():
        results.extend(tracker_results)
    return results
```

- [ ] **Step 2: Modify streaming to yield individual results**

Update `search_results_stream` to emit each result as it arrives:

```
# Track results seen per search
seen_hashes = set()

# In the polling loop, check for new individual results
live_results = orchestrator.get_live_results(search_id)
for result in live_results:
    if result.hash not in seen_hashes:
        seen_hashes.add(result.hash)
        yield SSEHandler.format_event(
            event="result_found",
            data={"result": {"name": result.name, "seeds": result.seeds, "
            event_id=search_id,
        )
```

- [ ] **Step 3: Run tests**

Run: `pytest tests/unit/test_streaming.py -v -k "results_stream"`
Expected: PASS

Task 3: Fix SSE endpoint to return actual results

Files: - Modify: `download-proxy/src/api/routes.py` (lines 235-241)

- [ ] **Step 1: Check current /search/stream/{search_id} endpoint**

The endpoint exists but may not be properly connected to the frontend. Verify it works:

```
curl -N http://localhost:7187/api/v1/search/stream/<search-id>
# Should stream results as SSE events
```

- [ ] **Step 2: Run tests**

Run: `pytest tests/unit/test_streaming.py -v` Expected: PASS

Task 4: Make frontend use SSE endpoint for live updates

Files: - Modify: `download-proxy/src/ui/templates/dashboard.html` (lines 463-520)

- ☐ **Step 1: Add EventSource for streaming results**

Replace polling with SSE in the search handler:

```
function streamResults(searchId) {
    if (!searchId) return;

    var eventSource = new EventSource('/api/v1/search/stream/' + searchId)

    eventSource.addEventListener('search_start', function(e) {
        console.log('Search started:', JSON.parse(e.data));
    });

    eventSource.addEventListener('result_found', function(e) {
        var result = JSON.parse(e.data).result;
        addResultToTable(result);  // Add single result immediately
        updateStatus('Found ' + totalResults + ' results...');
    });

    eventSource.addEventListener('results_update', function(e) {
        var data = JSON.parse(e.data);
        updateStatus('Found ' + data.total_results + ' results...');
    });

    eventSource.addEventListener('search_complete', function(e) {
        var data = JSON.parse(e.data);
        eventSource.close();
        loadStats();
    });

    eventSource.onerror = function() {
        eventSource.close();
    };
}
```

- ☐ **Step 2: Replace pollResults with streamResults**

In the search handler after getting search_id:

```
// OLD CODE - polling
if (_isSearching) {
    pollResults();
}

// NEW CODE - streaming
if (_isSearching) {
```

```
    streamResults(_searchId);
}
```

- [ ] **Step 3: Verify tests pass**

Run: `pytest tests/integration/test_ui_quick.py -v` Expected: PASS

Task 5: Integration tests for end-to-end streaming

Files: - Create: `tests/integration/test_realtime_streaming.py`

- [ ] **Step 1: Write comprehensive streaming tests**

```
def test_search_streams_results_in_realtime(self):
    """Search should stream results as they are found, not at the end."""
    # 1. Start a search query
    # 2. Connect to /search/stream/{search_id}
    # 3. Verify results come in via SSE BEFORE search completes
    # 4. Results should appear incrementally, not all at once
```

- [ ] **Step 2: Run tests**

Run: `pytest tests/integration/test_realtime_streaming.py -v` Expected: PASS

- [ ] **Step 3: Commit**

Expected Improvements

1. **Immediate feedback** - Users see results within seconds, not at the end of search (which can take 30+ seconds)
2. **Better UX** - Results table populates as results arrive
3. **Status updates** - â€œFound X results from RuTrackerâ€¦ etc. as trackers complete
4. **Reliability** - If one tracker fails, others continue working

Testing Commands

```
# Run unit tests
pytest tests/unit/test_streaming.py -v

# Run integration tests
pytest tests/integration/test_realtime_streaming.py -v --import-mode=impor

# Quick UI test
pytest tests/integration/test_ui_quick.py -v

# Manual verification
```

```
# 1. Open dashboard at http://localhost:7187
# 2. Search for something
# 3. Watch results appear in real-time as they arrive
```